

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

INGENIERÍA MECATRÓNICA



**LABORATORIO : INTELIGENCIA ARTIFICIAL Y VISIÓN POR
COMPUTADORA**

1MTR56

AUTOMATIZACIÓN INDUSTRIAL INTELIGENTE B

Tabla de contenido

1	Inteligencia Artificial: Redes Neuronales Convolucionales	2
1	Introducción	2
2	Preparación del entorno en VS Code	2
3	Redes neuronales convolucionales	2
3.1	Capa convolucional	3
3.2	Función de activación	4
3.3	Capa de pooling	4
3.4	Capas densas y salida	5
4	Entrenamiento	5
5	Evaluación y regularización	5
6	Aumentación de datos	5
7	Experiencia guiada	6
7.1	Notebook 1: CNN desde cero	6
7.2	Notebook 2: aumentación de datos	7
7.3	Notebook 3: clasificador de tapitas	8
7.4	Cierre: discusión crítica	8
8	Cierre: el límite del entrenamiento desde cero	9
2	Visión por Computadora: Transfer Learning y Beckhoff	10
9	Introducción	10
10	Aprendizaje por transferencia	10
10.1	Motivación	10
10.2	Qué se transfiere	10
10.3	MobileNetV2 como backbone	11
10.4	Las dos fases	11
11	El dataset de tapitas	11
12	Protocolo ADS y comunicación con Beckhoff	11
12.1	Configuración previa en TwinCAT	11
12.2	La librería pyads	12
13	Integración: del clasificador a la acción del brazo	12
14	Experiencia guiada	14
15	Anexos	15

Parte 1

Inteligencia Artificial: Redes Neuronales Convolucionales

1 Introducción

Esta guía introduce los fundamentos de la *visión por computadora* basada en aprendizaje profundo y los aplica a un problema de clasificación de imágenes con *redes neuronales convolucionales* (CNN). Constituye la **Parte 1** del laboratorio: se entrena una CNN *desde cero*; la Parte 2 introducirá el aprendizaje por transferencia sobre el mismo dominio.

Materiales. Python 3.10 con el entorno del laboratorio; el dataset de botellas provisto en `data/botellas/`; y los tres notebooks de la carpeta `parte1_ia/` (Notebook 1 a Notebook 3). Cada notebook explica su código en detalle; esta guía es la referencia teórica y describe *qué* se hace y *cuánto* aporta a la evaluación.

2 Preparación del entorno en VS Code

El único requisito delicado es usar un entorno virtual con la **versión correcta de Python** (3.10 o 3.11): con otra versión, TensorFlow no se instala correctamente.

1. Abrir la carpeta del proyecto en **Visual Studio Code**.
2. *Command Palette* (Ctrl+Shift+P) → Python: Create Environment → Venv → elegir un intérprete **Python 3.10** o **3.11**. Cuando VS Code lo solicite, marcar `requirements.txt` para que instale las dependencias en ese entorno.
3. En cada notebook, Select Kernel (arriba a la derecha) y elegir el entorno `.venv` recién creado.

Con el entorno correcto seleccionado, las celdas de `import` de los notebooks funcionan directamente (las dependencias ya quedaron instaladas al crear el entorno). Para confirmar la versión:

```
import sys, tensorflow as tf
print(sys.version)      # 3.10.x o 3.11.x
print(tf.__version__)   # 2.20.0
```

3 Redes neuronales convolucionales

Una CNN clasifica imágenes aprendiendo, capa por capa, a transformar los píxeles en características cada vez más abstractas hasta producir una probabilidad por clase. A diferencia de los métodos clásicos, no se diseñan los descriptores a mano: la red

los aprende sola durante el entrenamiento. Una arquitectura típica (Figura 1) alterna bloques de *convolución* y *pooling* —que extraen características— seguidos de *capas densas* que clasifican.

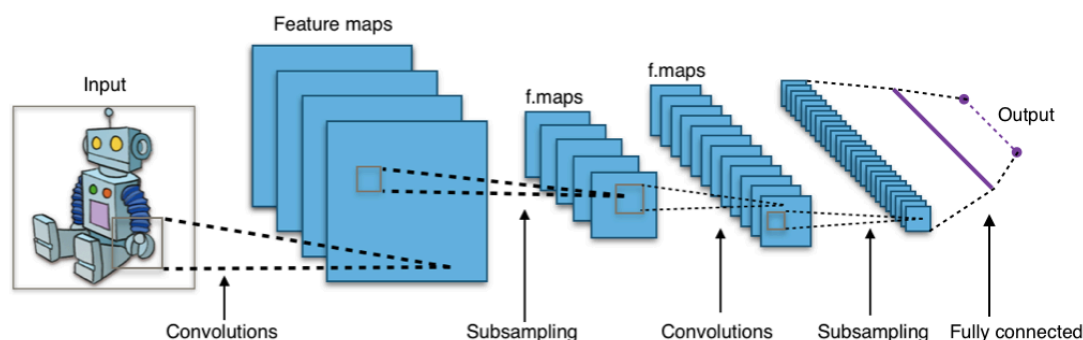


Figura 1. Arquitectura típica de una CNN: bloques de convolución y pooling extraen características de complejidad creciente; las capas densas producen la clasificación. Fuente: Wikimedia Commons (CC BY-SA).

3.1 Capa convolucional

La convolución desliza pequeños *filtros* (o *kernels*, p. ej. 3×3) sobre la imagen; en cada posición calcula un producto entre el filtro y la región que cubre, generando un *mapa de características* que resalta dónde aparece ese patrón (Figura 2). Los valores de los filtros son los parámetros que la red aprende. Tres hiperparámetros gobiernan la operación:

- **Tamaño del kernel:** la ventana del filtro (3×3 es lo más común); kernels mayores capturan patrones más extensos a mayor costo.
- **Stride (paso):** cuántos píxeles se desplaza el filtro. Stride 1 conserva la resolución; stride 2 la reduce a la mitad.
- **Padding (relleno):** añadir ceros en el borde. Con padding='same' la salida mantiene el tamaño de entrada; con 'valid' se encoge.

Cada capa usa *varios* filtros (32, 64, 128...): las primeras detectan bordes y texturas; las profundas, combinaciones más complejas (formas, partes de objetos).

Entrada (5 × 5)	Kernel Sobel-x (3 × 3)	Feature map (3 × 3)																																											
<table><tr><td>195</td><td>45</td><td>87</td><td>171</td><td>123</td></tr><tr><td>112</td><td>162</td><td>43</td><td>92</td><td>109</td></tr><tr><td>130</td><td>62</td><td>156</td><td>187</td><td>88</td></tr><tr><td>196</td><td>147</td><td>155</td><td>192</td><td>20</td></tr><tr><td>95</td><td>75</td><td>26</td><td>39</td><td>64</td></tr></table>	195	45	87	171	123	112	162	43	92	109	130	62	156	187	88	196	147	155	192	20	95	75	26	39	64	<table><tr><td>-1</td><td>+0</td><td>+1</td></tr><tr><td>-2</td><td>+0</td><td>+2</td></tr><tr><td>-1</td><td>+0</td><td>+1</td></tr></table>	-1	+0	+1	-2	+0	+2	-1	+0	+1	<table><tr><td>-220</td><td>111</td><td>100</td></tr><tr><td>-58</td><td>225</td><td>-205</td></tr><tr><td>-125</td><td>179</td><td>100</td></tr></table>	-220	111	100	-58	225	-205	-125	179	100
195	45	87	171	123																																									
112	162	43	92	109																																									
130	62	156	187	88																																									
196	147	155	192	20																																									
95	75	26	39	64																																									
-1	+0	+1																																											
-2	+0	+2																																											
-1	+0	+1																																											
-220	111	100																																											
-58	225	-205																																											
-125	179	100																																											

Figura 2. Operación de convolución: un filtro recorre la imagen y produce un mapa de características. Fuente: Stanford CS231n.

3.2 Función de activación

Tras cada convolución se aplica una activación no lineal. La más usada es **ReLU** ($\max(0, x)$): es rápida, mitiga el desvanecimiento del gradiente y aporta la no linealidad sin la cual la red entera colapsaría a una sola transformación lineal. Sigmoid y tanh se reservan para casos específicos; la capa de salida usa *softmax* (Figura 3).

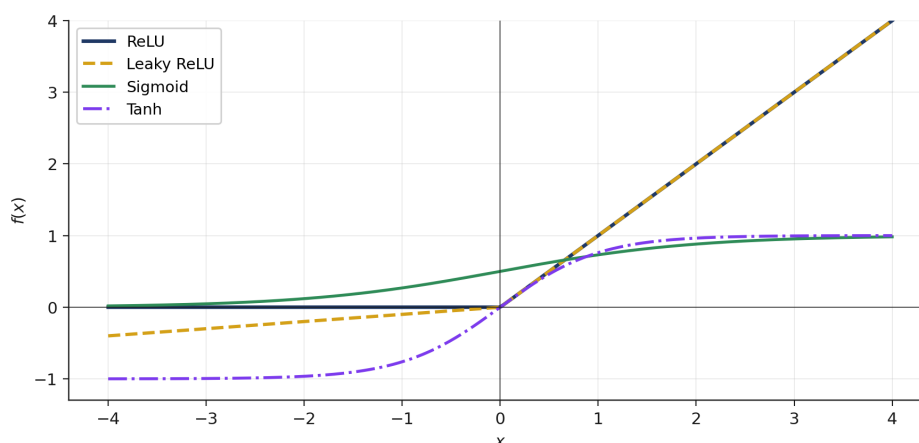


Figura 3. Funciones de activación habituales. *ReLU* es la más usada en capas convolucionales. Fuente: elaboración propia.

3.3 Capa de pooling

El *pooling* reduce la resolución de los mapas de características: baja el cómputo y aporta cierta invarianza a pequeños desplazamientos. Recorre el mapa en ventanas (típicamente 2×2) y resume cada una en un valor (Figura 4). Hay dos tipos:

- **Max pooling:** toma el *máximo* de cada ventana. Conserva la respuesta más fuerte —la presencia del patrón— y descarta el resto; es el más usado en clasificación porque resalta las características dominantes.
- **Average pooling:** toma el *promedio* de la ventana. Suaviza y conserva información de fondo; su variante *Global Average Pooling* resume cada mapa completo en un solo número al final de la red (lo usa MobileNetV2 en la Parte 2).

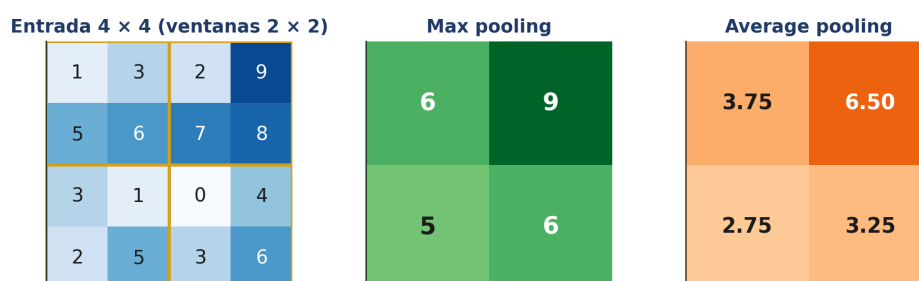


Figura 4. Max pooling: cada región se resume por su valor máximo, reduciendo la resolución. Fuente: elaboración propia.

3.4 Capas densas y salida

Tras los bloques convolucionales, las características se aplanan (*Flatten* o *global pooling*) y pasan a una o más **capas densas** que combinan toda la información. La **capa de salida** tiene tantas neuronas como clases y usa *softmax*, que convierte sus valores en probabilidades que suman 1; la clase predicha es la de mayor probabilidad, y ese valor es la *confianza*.

4 Entrenamiento

Entrenar es ajustar los filtros y pesos para minimizar el error. Cada paso tiene tres fases:

1. **Forward**: la imagen atraviesa la red y produce una predicción.
2. **Pérdida**: se compara con la etiqueta real. En clasificación multiclase se usa *categorical crossentropy*, que penaliza con fuerza predecir con alta confianza la clase equivocada.
3. **Backpropagation**: se calcula cuánto aportó cada peso al error (gradiente) y un **optimizador** (por defecto *Adam*) los ajusta.

Las imágenes no se procesan de una en una ni todas juntas, sino en **batches** (lotes, p. ej. 32 imágenes): la red predice el lote, promedia su pérdida y actualiza los pesos una vez. Un *batch size* grande estabiliza el gradiente pero consume más memoria. Una **época** es una pasada completa por todo el dataset; se entrena por varias épocas. El **learning rate** fija el tamaño de cada ajuste: muy alto oscila, muy bajo es lento.

5 Evaluación y regularización

El modelo se evalúa sobre un conjunto de **validación** independiente. Dos métricas clave: la **accuracy** (proporción de aciertos) y la **matriz de confusión**, que muestra no solo cuánto acierta sino *dónde* se confunde (qué clases mezcla).

Si la accuracy de entrenamiento es alta pero la de validación es baja, hay **overfitting**: la red memoriza en vez de generalizar. Defensas habituales:

- **Dropout**: durante el entrenamiento apaga al azar un porcentaje de neuronas, forzando a la red a no depender de unas pocas.
- **Batch normalization**: normaliza las activaciones de cada batch, lo que estabiliza y acelera el entrenamiento.
- **Data augmentation** (siguiente sección).

6 Aumentación de datos

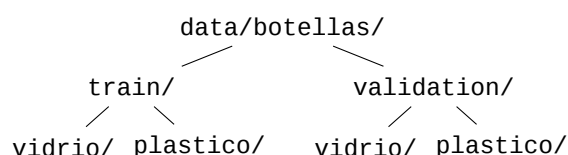
La *data augmentation* genera variaciones de las imágenes de entrenamiento — rotaciones, desplazamientos, cambios de brillo, zoom, volteos— para que el modelo

vea más diversidad y generalice mejor con pocos datos. Es la primera defensa contra el overfitting sin tocar la arquitectura, y se aplica **solo al entrenamiento**: la validación debe reflejar datos reales, sin alterar.

7 Experiencia guiada

El trabajo se desarrolla en tres notebooks que se ejecutan y completan de principio a fin, llenando las celdas marcadas como «completar» y respondiendo por escrito las preguntas conceptuales. Como los notebooks ya vienen implementados, **la nota evalúa la comprensión**: las respuestas a las preguntas conceptuales y la interpretación y justificación de los resultados, no la simple ejecución. La rúbrica completa está en la *Ficha de Evaluación*: la Parte 1 aporta **10 puntos** y la Parte 2 otros 10, para una nota sobre 20.

Los Notebooks 1 y 2 usan el dataset de botellas (vidrio/plástico) provisto; el Notebook 3 usa un dataset de tapitas construido por la propia pareja. El dataset se organiza en subcarpetas por clase dentro de train/ y validation/, de modo que ImageDataGenerator infiera las etiquetas automáticamente:



7.1 Notebook 1: CNN desde cero

Recorre el ciclo completo de una CNN entrenada desde cero: carga del dataset, definición y compilación de la arquitectura (Figura 5: bloques Conv2D + MaxPooling2D, Flatten, capa densa con *dropout* y salida *softmax*), entrenamiento, evaluación con matriz de confusión y predicción sobre imágenes propias (Figuras 6 y 7).

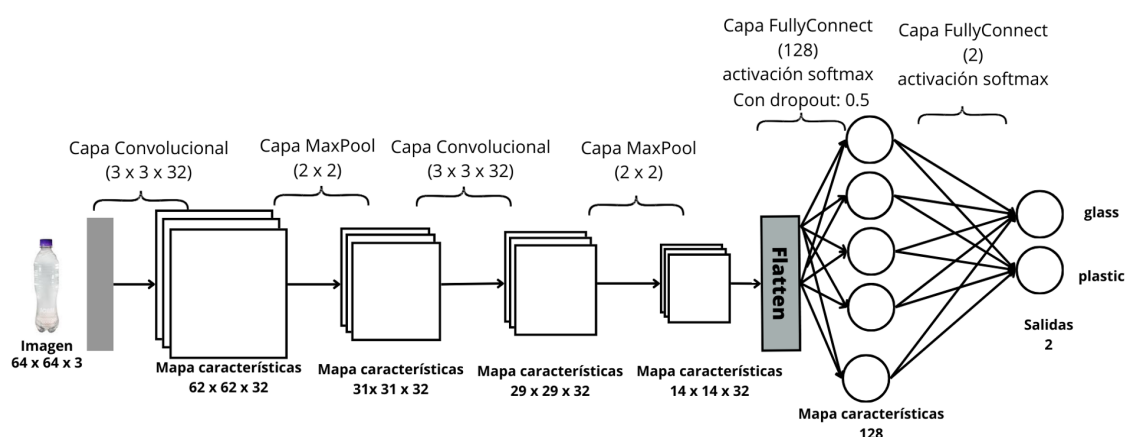


Figura 5. Arquitectura de la CNN del laboratorio. Fuente: elaboración propia (NN-SVG).

Etapa (secciones del notebook)	Qué se realiza	Puntos
1. Arquitectura y entrenamiento (secciones 3–6)	Definir y compilar la CNN, entrenar y analizar las curvas de <i>loss/accuracy</i> y la matriz de confusión sobre validación.	1.5
2. Evaluación con datos propios (sección 7)	Clasificar cuatro fotografías propias en condiciones variadas e identificar aciertos y fallos.	1.5

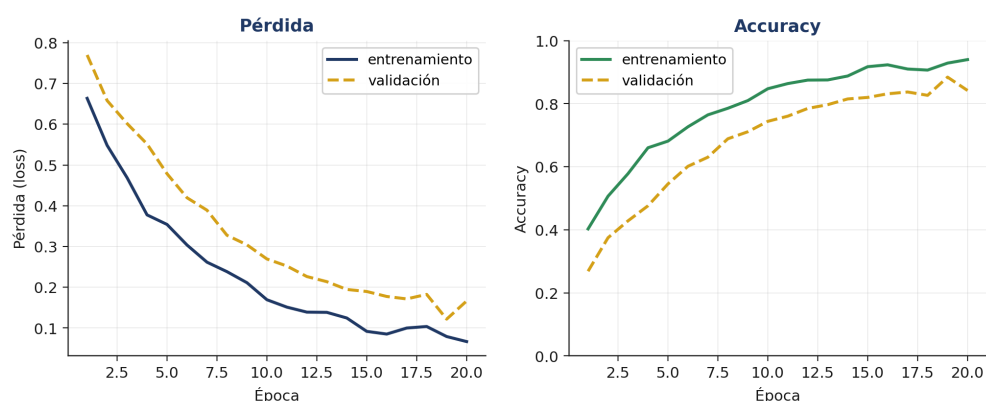


Figura 6. Ejemplo de curvas de aprendizaje sobre el dataset de botellas (resultado ilustrativo). Fuente: elaboración propia.

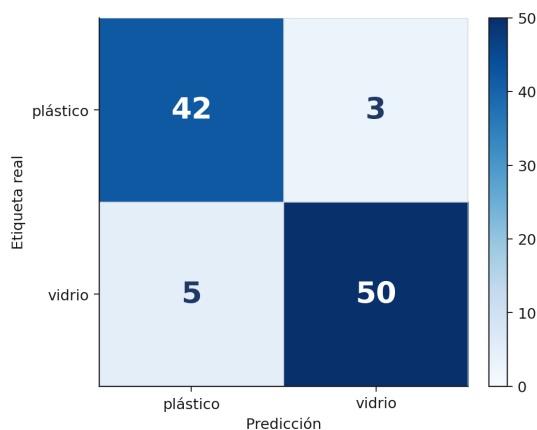


Figura 7. Ejemplo de matriz de confusión sobre validación (resultado ilustrativo); la diagonal son aciertos. Fuente: elaboración propia.

7.2 Notebook 2: aumentación de datos

Retoma el clasificador anterior para evidenciar y mitigar el *overfitting*: reproduce el sobreajuste del modelo base, selecciona transformaciones de aumentación pertinentes, reentrena y compara el desempeño frente al modelo sin aumentar.

Etapa (secciones del notebook)	Qué se realiza	Puntos
3. Data augmentation (secciones 2–5)	Diagnosticar el <i>overfitting</i> del modelo base, seleccionar transformaciones justificadas, reentrenar y comparar curvas frente al modelo base.	2.5

7.3 Notebook 3: clasificador de tapitas

Introduce un dataset propio: tapitas de colores (**rojo, azul, amarillo**) más una clase fondo (imágenes sin tapita). Se graba un video corto por color; el notebook extrae los *frames* a `data/tapitas/<clase>/` (descartando los borrosos), divide en *train/validation* y entrena una CNN desde cero. El modelo se exporta a `modelos/` para reutilizarlo en la Parte 2.

Protocolo de captura: iluminación y fondos variados, distancia 10–40 cm, ángulos diversos, una sola tapita por imagen y al menos 50 muestras por clase.

Etapa (pasos del notebook)	Qué se realiza	Puntos
4. Captura y preparación del dataset (pasos 1–2)	Grabar un video por color, extraer los frames a <code>data/tapitas/</code> y reportar el conteo por clase.	1.0
5. Entrenamiento y evaluación CNN (pasos 3–4)	Dividir <i>train/validation</i> , entrenar la CNN, analizar curvas y matriz de confusión, y exportar el modelo.	2.0

7.4 Cierre: discusión crítica

A partir de los resultados se plantean al menos dos hipótesis técnicas sobre las causas de los errores (sesgos del dataset, sensibilidad a la iluminación, límites de la arquitectura) y se proponen dos estrategias de mejora sin cambiar la arquitectura. Esta discusión se documenta en la Ficha de Evaluación (**1.5 puntos**).

Distribución de la nota de la Parte 1	Puntos
Notebook 1 — <code>notebook1_cnn_desde_cero.ipynb</code>	3.0
Notebook 2 — <code>notebook2_data_augmentation.ipynb</code>	2.5
Notebook 3 — <code>notebook3_clasificador_tapitas.ipynb</code>	3.0
Discusión crítica	1.5
Total Parte 1	10.0

8 Cierre: el límite del entrenamiento desde cero

La evaluación sobre datos propios suele revelar un resultado aleccionador: el modelo alcanza buena *accuracy* en validación, pero **falla ante condiciones reales** de iluminación, fondo o ángulo distintas a las del entrenamiento. La aumentación (Notebook 2) mitiga el problema pero no lo resuelve: una CNN entrenada desde cero necesita muchas más imágenes de las que es razonable capturar a mano.

Ese es el límite que motiva la **Parte 2**: en lugar de aprender todos los filtros desde cero, se parte de una red ya entrenada sobre millones de imágenes y se reutiliza su conocimiento —*aprendizaje por transferencia*—, obteniendo clasificadores robustos con apenas unos cientos de imágenes por clase. Sobre esa técnica se construye el sistema de control de calidad de la segunda parte, reutilizando el dataset de tapitas del Notebook 3.

Parte 2

Visión por Computadora: Transfer Learning y Beckhoff

9 Introducción

Esta es la **Parte 2** del laboratorio: visión por computadora aplicada al control de calidad. Se reutiliza el dataset de tapitas de la Parte 1, pero ahora el modelo se entrena con *aprendizaje por transferencia* (MobileNetV2) y el resultado se envía a un IPC **Beckhoff** por el protocolo **ADS** para accionar un brazo robótico.

Materiales. Los tres notebooks de parte2_vision/ (Notebook 4 a Notebook 6); la librería pyads; y un proyecto TwinCAT con el IPC Beckhoff.

10 Aprendizaje por transferencia

10.1 Motivación

Entrenar una CNN desde cero exige miles de imágenes por clase y mucho cómputo; con un dataset pequeño, como el de tapitas, sobreajusta enseguida. El *aprendizaje por transferencia* resuelve esto reutilizando una red ya entrenada sobre un dataset enorme.

10.2 Qué se transfiere

Las primeras capas de una CNN aprenden detectores *genéricos* —bordes, texturas, colores— útiles para casi cualquier imagen; solo las últimas son específicas de la tarea original. Por eso se conserva el cuerpo de la red —el *backbone*— y se reemplaza únicamente la parte final por un clasificador para las clases del problema (Figura 8).

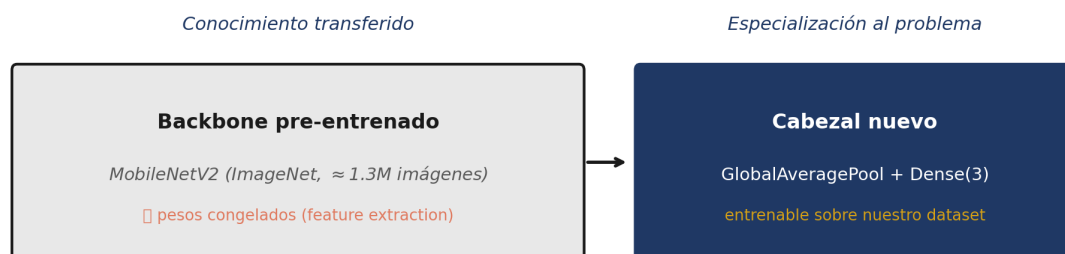


Figura 8. Aprendizaje por transferencia: se reutiliza el *backbone* preentrenado y se sustituye el clasificador final. Fuente: elaboración propia.

10.3 MobileNetV2 como backbone

Se usa **MobileNetV2**, preentrenada sobre **ImageNet** (1.4 millones de imágenes, 1000 clases). Es ligera (alrededor de 3.5 millones de parámetros) porque emplea *convoluciones separables en profundidad*, que descomponen la convolución estándar en dos pasos más baratos. Esa eficiencia permite ejecutarla en CPU o equipos embebidos, como exige un sistema de control de calidad en planta.

10.4 Las dos fases

1. **Feature extraction**: se *congela* el backbone (sus pesos no cambian) y se entrena solo un cabezal nuevo —un *Global Average Pooling* seguido de una capa densa *softmax* con las clases de tapitas—. Es rápido y suele bastar.
2. **Fine-tuning**: se *descongelan* las últimas capas del backbone y se reentrena con un *learning rate* muy pequeño. El LR bajo es crítico: uno grande destruiría el conocimiento de ImageNet (*olvido catastrófico*).

Con muy pocos datos basta el *feature extraction*; el *fine-tuning* ayuda cuando el dominio (tapitas, iluminación industrial) difiere del de ImageNet, a cambio de más riesgo de sobreajuste. El resultado es un clasificador robusto con apenas unos cientos de imágenes por clase, muy por encima de la CNN desde cero del Notebook 3.

11 El dataset de tapitas

Se reutiliza el dataset construido en la Parte 1 (Notebook 3): cuatro clases en orden alfabético —amarillo, azul, fondo, rojo—, con la división 80 % entrenamiento / 20 % validación. La clase fondo (imágenes sin tapita) es defensiva: evita que el modelo fuerce un color cuando no hay objeto y **no activa ninguna salida** del PLC. Si el dataset resultó escaso, conviene ampliarlo con más variación de iluminación, fondo y ángulo.

12 Protocolo ADS y comunicación con Beckhoff

El **ADS** (*Automation Device Specification*) es el protocolo de Beckhoff para intercambiar datos con el runtime TwinCAT. Permite acceder a las variables del PLC por su **nombre simbólico** (dentro de una *Global Variable List*, GVL), sobre TCP/IP y desde cualquier lenguaje; desde Python se usa la librería pyads (Figura 9).

12.1 Configuración previa en TwinCAT

Para que Python pueda escribir en el IPC, del lado TwinCAT se debe:

1. Declarar en una GVL llamada VARIABLES tres variables BOOL: bRojo, bAzul, bAmarillo.
2. Configurar una *route* ADS entre la PC y el IPC (AMS Net ID y puerto 851).
3. Construir un **HMI** en TwinCAT con una **lámpara por color**, cada una enlazada a su variable BOOL, para verificar visualmente que el clasificador escribe la variable

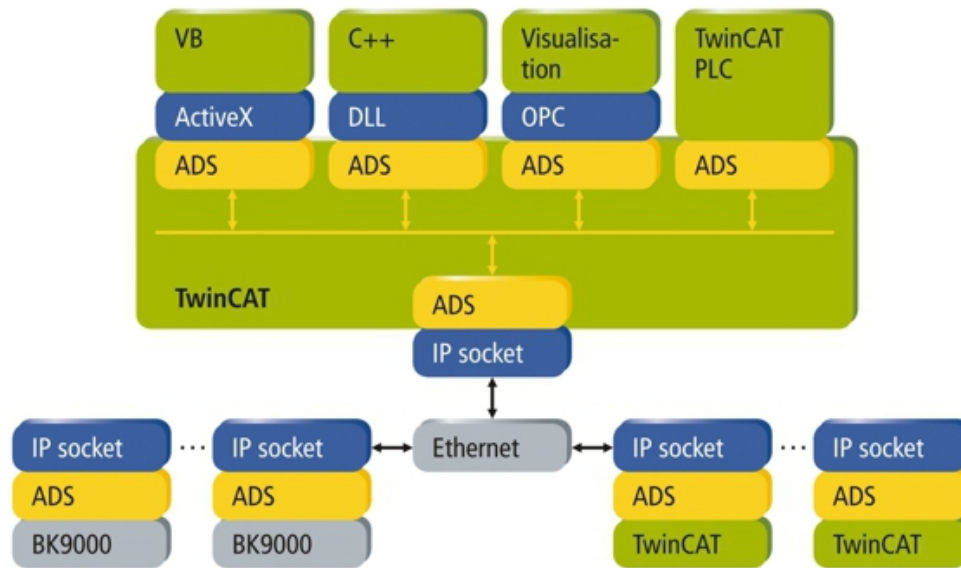


Figura 9. Arquitectura del protocolo ADS: las aplicaciones cliente acceden a las variables del PLC a través del *router* de TwinCAT. Fuente: Beckhoff Automation.

correcta.

12.2 La librería pyads

El patrón básico de conexión y escritura se muestra en el Script 1. Los nombres en `write_by_name()` deben coincidir *exactamente* con los de la GVL (mayúsculas incluidas).

Script 1. Conexión y escritura básica con pyads.

```
import pyads

ipc = pyads.Connection('192.168.57.231.1.1', 851)  # AMS Net ID del IPC
ipc.open()

ipc.write_by_name('VARIABLES.bRojo',      True,  pyads.PLCTYPE_BOOL)
ipc.write_by_name('VARIABLES.bAzul',      False, pyads.PLCTYPE_BOOL)
ipc.write_by_name('VARIABLES.bAmarillo',  False, pyads.PLCTYPE_BOOL)

ipc.close()
```

13 Integración: del clasificador a la acción del brazo

El objetivo final es cerrar el lazo: lo que la cámara ve se traduce en una acción del brazo. En cada *frame*, el programa repite cinco pasos:

1. **Capturar** el *frame* de la cámara.
2. **Preprocesar** la imagen igual que en el entrenamiento (redimensionar a 128 y normalizar con `preprocess_input` de MobileNetV2); si el preprocesamiento no

coincide con el del entrenamiento, las predicciones serán erróneas.

3. **Predecir** la clase y su confianza con el modelo entrenado.
4. **Decidir**: si la confianza supera el umbral (por defecto 0.6) y la clase es un color, se elige ese color; si es fondo o la confianza es baja, no se actúa.
5. **Comunicar**: se escribe True en la variable BOOL del color elegido y False en las otras dos. El PLC detecta el flanco ascendente, activa la salida digital y el brazo deposita la tapita en su zona.

El Script 2 implementa exactamente estos cinco pasos; conviene leerlo identificando dónde ocurre cada uno (los comentarios lo señalan).

Script 2. Bucle principal: captura, predicción y escritura ADS.

```
import cv2, numpy as np, pyads
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

UMBRAL = 0.6
IMG_SIZE = 128                                     # igual que al entrenar (NB4)
)
CLASES = ['amarillo', 'azul', 'fondo', 'rojo']      # orden alfabetico
VAR_POR_CLASE = {                                  # 'fondo' no activa ninguna
    salida
    'rojo': 'VARIABLES.bRojo',
    'azul': 'VARIABLES.bAzul',
    'amarillo': 'VARIABLES.bAmarillo',
}

modelo = load_model('modelo_tapitas.h5')
ipc = pyads.Connection('192.168.57.231.1.1', 851)
ipc.open()

cam = cv2.VideoCapture(0)
try:
    while True:
        ok, frame = cam.read()
        if not ok:
            break
        # Preprocesamiento MobileNetV2: BGR->RGB, resize y normalizar a [-1,
        1]
        img = cv2.cvtColor(cv2.resize(frame, (IMG_SIZE, IMG_SIZE)), cv2.
            COLOR_BGR2RGB)
        x = np.expand_dims(preprocess_input(img.astype('float32')), axis=0)

        pred = modelo.predict(x, verbose=0)[0]
        idx = int(np.argmax(pred))
        clase, confianza = CLASES[idx], float(pred[idx])

        # Activa el BOOL del color detectado si supera el umbral; el resto en
        False.
        activo = VAR_POR_CLASE.get(clase) if confianza >= UMBRAL else None
        for var in VAR_POR_CLASE.values():
            ipc.write_by_name(var, var == activo, pyads.PLCTYPE_BOOL)
```

```
cv2.putText(frame, f"{clase} ({confianza:.2f})", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
cv2.imshow('Clasificador', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
finally:
    cam.release(); cv2.destroyAllWindows()
    for var in VAR_POR_CLASE.values(): # apagar todo al salir
        ipc.write_by_name(var, False, pyads.PLCTYPE_BOOL)
    ipc.close()
```

14 Experiencia guiada

El trabajo se realiza en los tres notebooks de la Parte 2. Como ya están implementados, **la nota evalúa la comprensión**: las respuestas a las preguntas conceptuales y la interpretación de los resultados, no la simple ejecución. Por eso el entrenamiento —que es casi automático— pesa poco, y la comunicación real con Beckhoff y la inferencia en vivo pesan más. La Parte 2 aporta **10 puntos**.

El HMI en TwinCAT es obligatorio. Cada pareja debe construir una visualización con una **lámpara por color** (bRojo, bAzul, bAmarillo): es ahí donde se demuestra, en tiempo real, la comunicación de las predicciones con el PLC —cuando el clasificador detecta un color, su lámpara se enciende—. Sin el HMI no es posible evidenciar la Actividad 2 ni la demostración integrada.

Actividad (herramienta)	Qué se realiza	Puntos
1. Entrenamiento por transferencia (notebook4)	Entrenar (<i>feature extraction</i> + <i>fine-tuning</i>) e interpretar los resultados: comparar curvas y matriz de confusión y justificar la mejora frente a la CNN del Notebook 3. El entrenamiento es automático; se evalúa el análisis.	2.0
2. Comunicación con Beckhoff (TwinCAT + pyads, NB6 etapa 3)	Declarar las tres BOOL en la GVL, construir el HMI (obligatorio) con una lámpara por color y enviar valores booleanos desde Python (Script 1); verificar el encendido de cada lámpara.	2.5
3. Predicción de una imagen → PLC (notebook6 etapa 4)	Clasificar una imagen cargada y enviar el color detectado al PLC; explicar el rol del umbral de confianza y qué ocurre con la clase fondo.	2.0
4. Inferencia en tiempo real → PLC (notebook6 etapa 5)	Ejecutar el bucle con la cámara en vivo (Script 2): cada color detectado activa su BOOL y dispara la acción del brazo. Grabar el video de la demostración.	3.0
5. Discusión final	Reflexionar sobre cómo aplicar el sistema en el proyecto integrador de automatización.	0.5
Total Parte 2		10.0

15 Anexos

Anexo B1. Variables del proyecto TwinCAT

Variable	Tipo	Descripción
bRojo	BOOL	Verdadero al detectar una tapita roja sobre el umbral.
bAzul	BOOL	Verdadero al detectar una tapita azul sobre el umbral.
bAmarillo	BOOL	Verdadero al detectar una tapita amarilla sobre el umbral.

Deben declararse dentro de la GVL VARIABLES; el nombre de la GVL forma parte del identificador usado en `write_by_name()`.

Anexo B2. Diagnóstico de errores comunes

Error	Causa y solución
Missing ADS routes (7)	Falta la <i>route</i> ADS entre la PC y el IPC (open() no la valida; el error aparece al leer/escribir). Crearla en TwinCAT (bandeja) → Router → Edit Routes → Add Route.
AdsError: symbol not found	El nombre en write_by_name no coincide con la GVL. Verificar mayúsculas y el prefijo VARIABLES.
El brazo no se dispara	La variable se escribe pero el POU del PLC no actúa sobre ella. Verificar la lógica en TwinCAT.
Confianza alta pero clase incorrecta	Dataset poco diverso. Capturar más muestras en condiciones variadas.